

Aquí tienes un resumen breve de cada uno de los puntos clave tratados a lo largo de nuestra conversación:

- **Definición y función principal de un AST:** Un Árbol de Sintaxis Abstracta (AST, por sus siglas en inglés) es una estructura de datos jerárquica en forma de árbol que representa la estructura gramatical y lógica del código fuente, en lugar de tratarlo como texto plano.
- **Estructura básica de nodos:** Consta de un **Nodo Raíz** (que representa todo el programa), **Nodos Rama** (que representan estructuras compuestas como funciones, bucles o clases), **Nodos Hoja** para valores atómicos (literales, identificadores) y **Aristas** que mapean las relaciones padre-hijo.
- **Complejidad temporal de generación:** Para lenguajes de programación modernos con gramáticas deterministas, la generación de un AST desde cero se ejecuta en tiempo lineal $O(N)$ en función de la longitud del código.
- **Mantenimiento incremental del árbol:** Los analizadores sintácticos incrementales modernos (como Tree-sitter) actualizan un AST existente durante la edición en tiempo real en $O(M)$, reanalizando únicamente la región de código modificada (M) sin necesidad de reconstruir todo el archivo.
- **Niveles como alcance, no como significado:** La profundidad dentro de un AST refleja el alcance y la contención estructural (desde archivos hasta operadores), no el significado conceptual humano. Para capturar el "significado" o la similitud funcional del código, se requiere mapear los fragmentos del AST mediante modelos de *embedding* de código hacia espacios vectoriales multidimensionales.